

Syllabus (Spring 2025-2026)

DSA 201 Advanced Programming for Data Science

Instructor

İnanç Arın

e-mail: inancarin@sabanciuniv.edu

Course Description, Aim and Content

This course is designed to provide a comprehensive introduction to data science, while focusing on enhancing students' programming skills in Python. With this goal, the course aims to equip students with essential tools and techniques for data manipulation and visualization, and to introduce them to basic descriptive statistical analysis. Through a combination of theoretical concepts and practical applications, students will gain the skills needed to tackle real-world data science challenges.

Students will begin by practicing algorithmic problems to solidify their Python programming skills. They will then delve into data manipulation and preprocessing using powerful libraries such as NumPy and Pandas. The course also covers basic data visualization techniques using Matplotlib and Seaborn, as well as essential concepts in descriptive statistics. In the second part of the course, students will learn about object-oriented programming, basic data structures, and development of efficient algorithms.

Prerequisites

IF 100 - Computational Approaches to Problem Solving



Course Content¹

1. Warming Up with Algorithmic Problems in Python
 - a. Solving intermediate to advanced algorithmic challenges
 - b. Strengthening problem-solving and coding skills
 - c. Revisiting Dictionaries, introducing Sets (this can be done in chapter 7)
2. Data Manipulation and Preprocessing with NumPy and Pandas
 - a. Introduction to NumPy arrays and operations
 - b. Data manipulation with Pandas: loading, cleaning, and transforming data
 - c. Advanced data preprocessing techniques with Pandas
 - d. lambda, map, filter functions
3. Basic Data Visualization with Matplotlib and Seaborn libraries
 - a. Creating various types of plots: line, bar, histogram, scatter
 - b. Customizing plots for better insights
 - c. Other visualization techniques with Seaborn
4. Descriptive Statistics
 - a. Measures of central tendency: mean, median, trimmed mean
 - b. Measures of variability: variance, standard deviation, mean absolute deviation
 - c. Identifying outliers and understanding percentiles
 - d. Exploring relationships between variables: correlation
 - e. Visualizing statistical data
5. Object-Oriented Programming (OOP) in Python
 - a. Introduction to OOP concepts
 - b. Creating and using Python classes
 - c. Inheritance and polymorphism
6. Basic Data Structures (Part I)
 - a. Understanding and implementing stacks and queues
 - b. Practical applications of basic data structures

¹ The instructor may change the contents and the order of the topics during the semester.

- 
7. Understanding Program Efficiency
 - a. Analyzing algorithm efficiency
 - b. Big O notation and growth rates
 - c. Searching algorithms and their time complexity
 - d. Solving and interpreting common recurrence relations
 8. Sorting Algorithms
 - a. Comparison and analysis of different sorting techniques
 - b. Leveraging sorting for building efficient algorithms
 9. Data Structures (Part II)
 - a. Binary Heap
 - b. Linked List
 - c. Binary Search Tree
 - d. Real-world applications of these data structures

Grading²

- Recitation Attendance 4%
 - Take-Home Exams (THEs) 4%
 - Midterm Exam 40%
 - Final Exam 52%
- If your weighted exam average $((\text{midterm} * 0.40 + \text{final} * 0.52) / 0.92)$ is strictly below 30, you will automatically fail the course. However, achieving a weighted exam average of 30 or above does not guarantee that you will pass the course.
- All of the assigned THEs will be graded and taken into consideration in the overall grade. Each THE will have equal weight in overall grading. **If your Take Home Exam (THE) average falls below 40, you will automatically fail the course.** While the THE carries relatively low weight in your final grade, it's important to note that similar questions will appear on both the midterm and final exams.

² Please note that the grading schema is tentative and can be modified by the instructor as deemed necessary.

- 
- Recitation grade is based on your attendance. Students must attend their assigned recitation sections.
 - There won't be any make-up for THEs and missing recitations in any case, including medical health reports and official university activities.
 - If you miss all of the recitation attendances; then you will get an NA grade.
 - If you miss the midterm examination or the final examination, and if you do not take the make-up exam for that missing exam; then you will directly get an F grade.
 - There will be a single make-up exam at the end of the semester (after the final exam) and this exam will be comprehensive (i.e. all topics included). You are allowed to miss only one exam, regardless of whether you have a medical report. Missing more than one exam will result in an automatic failure of the course.
 - In order to take the make-up exam, students must submit their medical reports to the university health center to validate them and notify the instructor within the day of the exam to be missed. The ones having other excuses should also contact the instructor within the day of the exam to be missed, and then the instructor will decide whether these students are allowed to take the make-up exam.
 - The instructor has the right to have an oral interview for any grading item given in the syllabus. Students who will have the oral interview may be selected randomly or according to a suspicious situation observed by TAs or the instructors. After having the oral interview, the grade obtained by the student may change in a positive or a negative way.

Generative AI Policy

In this course, you may use generative AI tools (e.g., ChatGPT, Gemini, Copilot, Claude, etc.) **only as a learning aid**, not as a solution generator.

Allowed uses (learning support):

- Explaining concepts (e.g., Python syntax, pandas operations, OOP ideas)
- Interpreting error messages and suggesting debugging steps
- Providing small, isolated examples **not directly solving the assigned task**
- Discussing alternative approaches at a high level (without writing the final solution)

Not allowed (solution generation):

- Generating a full or near-complete solution for any assignment, take-home exercise, or coding task
- Rewriting your code into a finished solution based on the assignment prompt
- Producing final code/analysis/text that you submit as your own work
- Using AI to “fill in the missing parts” so that the submission becomes a complete answer

Disclosure requirement:

If you consult an AI tool, you must include a short note (as a comment in your code) in your submission stating **what you used it for** (e.g., “Used AI to understand a pandas error message and to brainstorm debugging steps”). This note does **not** give extra credit, but helps maintain transparency.

Accountability / verification:

You are responsible for understanding every part of your submission. The instructor and teaching assistants may ask you to explain your work (e.g., brief oral check, targeted questions, or follow-up tasks). If you cannot explain key parts of your submission, the grade may be adjusted, including receiving zero for that item.

Academic integrity:

Using generative AI to produce submitted solutions is treated as an academic integrity violation and will be handled according to the course and university policies.

Plagiarism Policy (Academic Integrity)

Plagiarism means presenting someone else's work as yours (including Generative AI's solution as explained above). This is a very serious and ethical problem. [This link](#) (please read it) is prepared in order to explain the actions that yield to plagiarism, and the sanctions against it. Most of the actions and sanctions mentioned in this part will also be valid for IF100, CS201 and CS204 courses as well; but there might be small differences for the rules among these courses.

When a plagiarism case is detected, sanctions are applied to all parties regardless of the actual source of the submission. These sanctions are as follows:

- For the midterm/final examinations,
 - students directly fail the course, even in the first offense.
- For THEs,
 - For the first time, all plagiarized submission owners receive a grade of 0 (zero),
 - For the second time, the student fails the course automatically.

References

- **"Python for Data Analysis" by Wes McKinney:** This book is written by the creator of the Pandas library and provides a comprehensive introduction to data analysis with Python. It covers data manipulation, cleaning, and visualization, using libraries like Pandas, NumPy, and Matplotlib.
<https://wesmckinney.com/book/>
- **"Python Data Science Handbook" by Jake VanderPlas (Chapter 2, 3, and 4 of the book):** This is an excellent resource that covers essential data science tools and techniques in Python. It delves into libraries such as NumPy, Pandas, Matplotlib, and more, providing practical examples and explanations.
<https://learning.oreilly.com/library/view/python-data-science/9781491912126/>

- **"Data Science from Scratch: First Principles with Python" by Joel Grus (Chapter 3 and chapter 5 of the book):** This book teaches data science fundamentals using Python. It is a good starting point for beginners as it builds from basic programming concepts to more complex data science topics.
<https://learning.oreilly.com/library/view/data-science-from/9781492041122/>
- **"Practical Statistics for Data Scientists: 50+ Essential Concepts Using R and Python" by Peter Bruce and Andrew Bruce (Chapter 1 of the book):** This book provides a practical introduction to statistical concepts used in data science, with examples in both R and Python. It is a good resource for understanding the statistical foundations of data science.
<https://learning.oreilly.com/library/view/practical-statistics-for/9781492072935/>
- **"Effective Python: 90 Specific Ways to Write Better Python" by Brett Slatkin (Only chapter 5 of the book):** This book offers practical advice and tips for writing more efficient and effective Python code, which is valuable for any data scientist looking to improve their coding skills.
<https://learning.oreilly.com/library/view/effective-python-90/9780134854717/>
- **Introduction to Python Programming and Data Structures by Y. Daniel Liang (Various chapters of the book)**

Announcements and SUCourse+

Students are responsible for all announcements made during the regular class meetings. Students should check their Sabanci University e-mail accounts (mymail@sabanciuniv.edu) and follow the SUCourse+ site (<https://sucourse.sabanciuniv.edu/plus/>) for this class regularly as they are responsible for all announcements and postings delivered through these sources.

Important Note: This document may be modified during the semester (Spring 2025-2026) due to unforeseen reasons.